ChaosMend

ML-Driven Self-Healing Microservices Platform

A distributed system that learns to heal itself by intentionally breaking

Author:	Swetha Sekar
Institution:	Trinity College Dublin
Program:	MSc Computer Science (Data Science)
Date:	March 2026
Version:	1.0

Table of Contents

1. Executive Summary

ChaosMend is a chaos engineering platform that combines distributed systems reliability engineering with machine learning to create self-healing microservices. The system intentionally induces failures (chaos engineering), uses ML models to detect anomalies and predict cascading failures, then automatically heals itself based on learned recovery patterns.

Key Innovations:

• **Proactive Chaos Engineering:** Randomly induces failures in production-like environments

• **ML-Based Anomaly Detection:** Uses Isolation Forest and LSTM autoencoders to detect unusual patterns

• **Predictive Failure Analysis:** Identifies cascading failure patterns before they occur

• **Automated Self-Healing:** Learns optimal recovery strategies and executes them automatically

• **Hybrid Deployment Model:** Starts with Docker Compose, migrates to Kubernetes

Component	Technology
Backend Services	Python 3.11 + FastAPI
Message Broker	Apache Kafka
Containerization	Docker → Kubernetes (Minikube)
Metrics & Monitoring	Prometheus + Grafana
Database	PostgreSQL
ML Framework	Scikit-learn, Prophet, LSTM

2. Real-World Use Case & Platform Overview

2.1 The Platform: Digital Payment Processing System

ChaosMend simulates a real-world **digital payment processing platform** similar to Stripe, Square, or PayPal. The system handles high-volume financial transactions with strict requirements for reliability, security, and compliance.

Platform Capabilities:

• Process credit card transactions in real-time

• Detect and prevent fraudulent transactions

• Send instant notifications to users and merchants

• Maintain transaction analytics and reporting

• Ensure 99.9% uptime despite system failures

• Scale automatically during high-traffic periods (Black Friday, holiday sales)

2.2 Transaction Flow Example

Scenario: Sarah purchases a $299 laptop from TechStore using the payment platform. **Step 1: Transaction Creation** • Sarah enters credit card details on TechStore's checkout page • Transaction Service receives request: {user_id: "sarah_123", amount: 299.00, merchant: "TechStore"} • Creates transaction record in PostgreSQL • Publishes "TransactionCreated" event to Kafka **Step 2: Risk Assessment** • Risk Service consumes the Kafka event • Checks: unusual location? unusual amount? velocity abuse? • Risk score: 0.15 (low risk, Sarah's normal shopping pattern) • Publishes "TransactionApproved" event **Step 3: Notifications** • Notification Service consumes "TransactionApproved" event • Sends email to Sarah: "Your payment of $299.00 to TechStore is approved" • Sends webhook to TechStore: "Payment confirmed, ship order #12345" **Step 4: Analytics** • Analytics Service updates real-time dashboard • Today's transaction volume: $1.2M (↑ 15%) • Average transaction value: $87.50 • Fraud rate: 0.3% **Total Processing Time:** 180ms (end-to-end)

2.3 What Can Go Wrong? (Why Chaos Engineering Matters)

In a real payment platform, failures happen constantly: **Scenario 1: Database Connection Loss** • Transaction Service can't connect to PostgreSQL • Without self-healing: All new transactions fail, users see errors • With ChaosMend: System detects DB failure, routes to backup, auto-reconnects **Scenario 2: Risk Service Pod Crash** • Risk Service crashes due to memory leak • Without self-healing: Transactions processed without fraud checks (dangerous!) • With ChaosMend: Anomaly detector sees rising memory usage, triggers restart before crash **Scenario 3: Kafka Consumer Lag** • Notification Service falls behind processing events • Without self-healing: Users don't receive payment confirmations for 10+ minutes • With ChaosMend: Detects lag spike, auto-scales notification consumers from 2 → 5 **Scenario 4: Cascading Failure** • Transaction Service overloaded → starts rejecting requests → users retry → more load → complete failure • Without self-healing: Entire platform goes down • With ChaosMend: ML predicts cascade pattern, scales Transaction Service proactively, adds rate limiting

3. Problem Statement

3.1 Current State of Microservices Reliability

Modern distributed systems fail in complex, unpredictable ways. Traditional approaches have significant limitations: **Manual Recovery:** Requires human intervention, mean time to recovery (MTTR) of 15-60 minutes **Reactive Monitoring:** Alerts fire AFTER failures occur, not before **Static Auto-Scaling:** Rules-based scaling (CPU > 80% $\rightarrow$ add pods) is too slow and wasteful **Unknown Failure Modes:** Teams don't know how their systems behave under real failure conditions until production incidents occur

3.2 The ChaosMend Approach

ChaosMend addresses these challenges through: 1. **Intentional Failure Injection:** Chaos agent randomly breaks services in controlled ways, discovering failure modes before they happen in production 2. **ML-Based Prediction:** Anomaly detection models learn normal behavior patterns and predict failures before they cascade 3. **Automated Healing:** Self-healing agent executes recovery actions (restart, scale, reroute) based on learned strategies 4. **Continuous Learning:** System tracks which healing actions work, improving over time

4. System Architecture

4.1 High-Level Architecture

The system consists of three layers: **Application Layer:** Business logic microservices (Transaction, Risk, Notification, Analytics) **Infrastructure Layer:** Kafka (events), PostgreSQL (data), Prometheus (metrics), Grafana (visualization) **Chaos & Healing Layer:** Chaos Agent, Anomaly Detector, Self-Healing Agent All services run in containers, initially on Docker Compose, then migrated to Kubernetes.

4.2 Component Descriptions

Component	Responsibility	Technology
Transaction Service	Create/manage transactions, publish events	FastAPI + PostgreSQL
Risk Service	Fraud detection, flag risky transactions	FastAPI + Kafka Consumer
Notification Service	Send alerts to users/merchants	FastAPI + Kafka Consumer
Analytics Service	Real-time metrics aggregation	FastAPI + Kafka Streams
Chaos Agent	Randomly inject failures (kill pods, add latency)	Python + Docker/K8s API
Anomaly Detector	ML-based pattern detection, predict failures	Scikit-learn + Prophet
Self-Healing Agent	Execute recovery actions automatically	Python + K8s API
Kafka	Event streaming backbone	Apache Kafka 7.5.0
Prometheus	Metrics collection & storage	Prometheus 2.45+
Grafana	Metrics visualization dashboards	Grafana 10.x

4.3 Event Flow Architecture

Normal Operation Flow: 1. User API → Transaction Service → Creates transaction in DB 2. Transaction Service → Publishes "TransactionCreated" event to Kafka 3. Kafka → Broadcasts event to all subscribers (Risk, Notification, Analytics) 4. Each service processes independently and may publish new events 5. All services emit metrics to Prometheus every 10 seconds **Chaos Engineering Flow:** 1. Chaos Agent (runs every 5 minutes) → Randomly selects action (kill pod, inject latency, etc.) 2. Chaos Agent → Executes action via Docker/K8s API 3. Chaos Agent → Publishes "ChaosEventOccurred" to Kafka 4. Target service → Crashes/slows down 5. Prometheus → Detects abnormal metrics **Detection & Healing Flow:** 1. Anomaly Detector (runs every 30 seconds) → Queries Prometheus for metrics 2. ML Model → Analyzes patterns, detects anomalies (CPU spike, error rate increase, etc.) 3. Anomaly Detector → Publishes "AnomalyDetected" event to Kafka 4. Self-Healing Agent → Consumes anomaly event 5. Self-Healing Agent → Queries healing strategy database (Q-table or experience replay) 6. Self-Healing Agent → Executes best action (restart pod, scale up, reroute traffic) 7. Self-Healing Agent → Monitors metrics for 2 minutes to verify success 8. Self-Healing Agent → Updates strategy table with result (+1 reward if success, -1 if failed)

5. Technical Specifications

5.1 Technology Stack Details

Programming Language: Python 3.11 **Web Framework:** FastAPI 0.104 (async support, automatic OpenAPI docs, Pydantic validation) **Message Broker:** Apache Kafka 7.5.0 (high-throughput event streaming, persistent logs) **Database:** PostgreSQL 15 (ACID compliance for financial transactions) **Containerization:** Docker 24.x (consistent environments across dev/staging/prod) **Orchestration:** Docker Compose (Phase 1-3) → Kubernetes 1.28+ Minikube (Phase 4-5) **Metrics:** Prometheus 2.45+ (time-series metrics, pull-based collection) **Visualization:** Grafana 10.x (real-time dashboards, alerting) **ML Libraries:** • Scikit-learn 1.3.2 (Isolation Forest for anomaly detection) • Prophet 1.1.5 (time-series forecasting) • TensorFlow/Keras (LSTM autoencoders for advanced anomaly detection) • Pandas 2.1.3 (data manipulation) • NumPy 1.26.2 (numerical operations) **Kubernetes Client:** kubernetes-python 28.1.0 (programmatic K8s control for chaos/healing) **Testing:** pytest 7.4.3, pytest-asyncio 0.21.1, pytest-cov 4.1.0

5.2 Key Dependencies Rationale

Dependency	Why This Choice?
FastAPI	Async support (critical for high-throughput), auto docs, Pydantic validation
Kafka	Industry standard at Uber/LinkedIn/Netflix, better than RabbitMQ for scale
PostgreSQL	ACID guarantees for financial data, better than MongoDB for transactions
Prometheus	Pull-based metrics (better than push), PromQL query language, K8s native
Scikit-learn	Battle-tested ML library, Isolation Forest is fast and accurate
Prophet	FB open-source, excellent for time-series with trends/seasonality

6. Development Phases (Hybrid Docker → Kubernetes)

The project follows a **hybrid deployment strategy**: start with Docker Compose for rapid development and learning, then migrate to Kubernetes to demonstrate production-readiness. This approach balances learning efficiency with resume impact.

Phase 1: Base Microservices (Docker) - Weeks 1-2

Goal: Build core application services and validate event-driven architecture **Deliverables:** • Transaction Service (FastAPI + PostgreSQL) • Risk Service (Kafka consumer, fraud scoring logic) • Notification Service (Kafka consumer, alert sending) • Analytics Service (optional - real-time metrics aggregation) • Kafka + Zookeeper running in Docker Compose • PostgreSQL database with schema • Shared Python packages (schemas, Kafka utilities) • End-to-end integration test **Success Criteria:** • Create transaction via API → Risk check → Notification sent • All services run via: docker-compose up • Events visible in Kafka UI • No Kubernetes required yet **Learning Focus:** • Event-driven architecture patterns • Kafka producer/consumer patterns • Async Python programming (FastAPI) • Docker networking and volumes • Database schema design

Phase 2: Metrics & Monitoring (Docker) - Week 2

Goal: Add observability layer for chaos engineering foundation **Deliverables:** • Prometheus added to docker-compose.yml • All services expose /metrics endpoint • Grafana dashboards created: - System Health (CPU, memory, request rate per service) - Transaction Metrics (volume, latency, error rate) - Kafka Metrics (consumer lag, throughput) • Basic alerting rules in Prometheus **Success Criteria:** • Grafana accessible at localhost:3000 • All service metrics visible in Prometheus • Dashboards auto-refresh every 10 seconds • Can manually kill a container and see metrics spike **Learning Focus:** • Prometheus metrics exposition • PromQL query language • Grafana dashboard design • Time-series data analysis

Phase 3: Chaos Engineering (Docker) - Week 3

Goal: Build chaos agent to induce failures systematically **Deliverables:** • Chaos Agent service (Python + Docker SDK) • Chaos actions implemented: - Random container kill (docker kill) - CPU stress (stress-ng container injection) - Memory stress (memory leak simulation) - Network latency (tc command via privileged container) • Chaos event logging to Kafka (chaos.events topic) • Chaos dashboard in Grafana • Configurable chaos schedule (e.g., every 5 minutes, 30% probability) **Success Criteria:** • Chaos agent runs automatically • Services recover from container kills (Docker restarts them) • Chaos events logged and visible in Grafana • Can trigger chaos manually via API: POST /chaos/trigger **Learning Focus:** • Docker SDK for Python • Failure injection techniques • Observing system behavior under stress • Manual recovery vs auto-recovery comparison

Phase 4: ML Anomaly Detection (Docker) - Week 4

Goal: Add ML-based anomaly detection to predict failures **Deliverables:** • Anomaly Detector service • ML models implemented: - Isolation Forest (fast baseline for outlier detection) - Prophet (time-series forecasting with trend/seasonality) - LSTM Autoencoder (optional - for advanced pattern recognition) • Metrics collection pipeline: Prometheus → Pandas DataFrame → ML Model • Anomaly scoring algorithm (0.0 = normal, 1.0 = severe anomaly) • Alert generation: Publishes "AnomalyDetected" events to Kafka • Anomaly dashboard in Grafana (detected anomalies timeline, false positive rate) **Features Monitored:** • CPU usage (per container) • Memory usage (per container) • Request rate (per service) • Error rate (per service) • Response latency p99 (per service) • Kafka consumer lag (per consumer group) **Success Criteria:** • Model detects container crash 30-60 seconds before it happens • False positive rate < 15% • Anomalies visible in Kafka UI and Grafana • Can tune detection sensitivity via config **Learning Focus:** • Time-series anomaly detection techniques • Feature engineering for metrics • Model evaluation (precision, recall, F1) • Balancing sensitivity vs false positives

Phase 5: Self-Healing (Docker) - Week 5

Goal: Implement automated healing based on anomaly detection **Deliverables:** • Self-Healing Agent service • Healing strategies: - Container restart (docker restart) - Container scaling (docker-compose scale service=N) - Manual intervention trigger (sends Slack alert for human review) • Strategy selection algorithm: - Simple: Rule-based (if CPU > 80% → scale up) - Advanced: Q-learning (track success rate of each action, choose best) • Healing action logging to Kafka (healing.actions topic) • Healing dashboard in Grafana: - Mean time to recovery (MTTR) - Healing success rate (%) - Action type distribution (restart vs scale vs manual) **Success Criteria:** • 80%+ of anomalies auto-heal without human intervention • MTTR < 2 minutes (from anomaly detection to full recovery) • No infinite restart loops (circuit breaker logic) • Healing actions explainable (logs show: detected X, tried Y, result Z) **Learning Focus:** • Reinforcement learning basics (state, action, reward) • Circuit breaker patterns • Graceful degradation strategies • Production-readiness thinking

Phase 6: Kubernetes Migration (Optional) - Week 6

Goal: Migrate working system from Docker to Kubernetes **Why Migrate?** • Resume keyword: "Deployed on Kubernetes" • Production patterns: Deployments, Services, ConfigMaps, Secrets • Auto-scaling: Horizontal Pod Autoscaler (HPA) • Advanced chaos: Pod deletion is more realistic than container kill **Deliverables:** • Kubernetes manifests for all services: - Deployments (with replicas=2 for resilience) - Services (ClusterIP for internal, LoadBalancer for external) - ConfigMaps (for configuration) - Secrets (for DB passwords, API keys) • Chaos Agent updated to use Kubernetes API (kubernetes-python library) - Pod deletion (kubectl delete pod) - Network policies (isolate pods) - Resource limits (CPU/memory throttling) • Self-Healing Agent updated to use K8s API - Scale replicas: PATCH Deployment - Restart pods: DELETE Pod (K8s creates new one) - Traffic rerouting: UPDATE Service selector • Prometheus + Grafana deployed on K8s • Horizontal Pod Autoscaler (HPA) configured • Minikube deployment working **Success Criteria:** • All services running on Minikube • Can deploy with: kubectl apply -f k8s/ • Chaos agent kills pods, K8s recreates them automatically • Self-healing agent scales deployments dynamically • Metrics still collected by Prometheus **Learning Focus:** • K8s architecture (Pods, Deployments, Services) • kubectl commands • K8s networking (DNS, ClusterIP, NodePort) • K8s API programmatic control • Production deployment patterns

Phase 7: Evaluation & Documentation - Week 7

Goal: Benchmark performance, document results, prepare for interviews **Deliverables:** • Performance benchmarks: - Baseline vs Chaos-enabled system (uptime %, MTTR) - Manual recovery vs Auto-healing (recovery time comparison) - Cost analysis (over-provisioned resources vs ML-optimized) • Experiment reports (saved in docs/experiments/) • Architecture diagrams (Lucidchart or draw.io) • API documentation (auto-generated Swagger/OpenAPI) • README with setup instructions • Demo video (5-10 minutes showing chaos → detection → healing) • Blog post / Medium article (optional) • Research paper draft (optional, if targeting publication) **Success Criteria:** • Can demonstrate full system in < 5 minutes • All code documented with docstrings • GitHub repository professional-quality • Ready to discuss in interviews **Learning Focus:** • Technical writing • Storytelling for interviews • Benchmarking methodology • Professional portfolio presentation

7. Features & Capabilities

7.1 Chaos Engineering Capabilities

Chaos Action	Description	Implementation
Container/Pod Kill	Randomly terminate containers	docker kill (Phase 1-5), kubectl delete pod (Phase 6)
CPU Stress	Max out CPU to 100%	stress-ng container, --cpu 0 --cpu-load 100
Memory Stress	Consume RAM until OOM	stress-ng --vm 1 --vm-bytes 1G
Network Latency	Add 100-500ms delay	tc qdisc add dev eth0 root netem delay 200ms
Network Partition	Isolate service from others	iptables rules or K8s NetworkPolicy
Disk I/O Stress	Slow down disk reads/writes	stress-ng --hdd 1 --hdd-bytes 1G

7.2 ML Anomaly Detection Models

Model	Use Case	Accuracy Expectation
Isolation Forest	Fast outlier detection, multi-dimensional	85-90% (baseline)
Prophet	Time-series with trends/seasonality	90-95% (if clear patterns)
LSTM Autoencoder	Complex temporal patterns	92-97% (advanced, slower)
One-Class SVM	Alternative to Isolation Forest	80-85% (comparison)

7.3 Self-Healing Strategies

Detected Issue	Healing Action	Expected Recovery Time
Container crash loop	Delete container/pod (force restart)	< 30 seconds
CPU > 80% for 3 min	Scale replicas +2	45-60 seconds
Memory > 85% for 2 min	Restart container/pod	< 45 seconds
Consumer lag > 1000 msgs	Scale consumer group +3	60-90 seconds
Error rate > 5%	Restart + circuit breaker	30-60 seconds
Response latency > 2x baseline	Scale + add cache layer	90-120 seconds

8. Metrics & Success Criteria

8.1 System Health Metrics

Metric	Target	Critical Threshold	Measurement Method
System Uptime	> 99.5%	< 95%	Prometheus up{job="service"} metric
Mean Time To Recovery (MTTR)	< 2 min	> 5 min	Time from anomaly detection to recovery
False Positive Rate	< 10%	> 20%	Manual validation of alerts
Detection Latency	< 30s	> 60s	Time from failure to anomaly detection
Healing Success Rate	> 80%	< 60%	(successful_healings / total_attempts) × 100

8.2 Minimum Viable Product (MVP) Checklist

✓ 3+ microservices communicating via Kafka
✓ All services containerized (Docker)
✓ Metrics exposed to Prometheus
✓ Grafana dashboards for system health
✓ Chaos agent can kill containers/pods randomly
✓ At least 2 chaos actions implemented (kill + one other)
✓ Chaos events logged to Kafka
✓ Anomaly detection model working (Isolation Forest minimum)
✓ Detects at least container crashes accurately
✓ Alerts generated on anomalies
✓ Self-healing agent can restart containers automatically
✓ At least one healing strategy implemented beyond restart
✓ Healing actions logged to Kafka
✓ Architecture diagram documenting system
✓ README with setup instructions
✓ Demo video (5-10 minutes) showing chaos → detect → heal flow

9. Data Models & API Specifications

9.1 Kafka Event Schemas

```
Topic: transaction.created { "transaction_id": "txn_abc123", "user_id":
"user_456", "amount": 299.00, "merchant": "TechStore", "currency": "USD",
"timestamp": "2025-03-11T14:30:00Z" } Topic: transaction.flagged {
"transaction_id": "txn_abc123", "risk_score": 0.85, "reason":
"unusual_location", "flagged_by": "risk-service", "timestamp":
"2025-03-11T14:30:02Z" } Topic: chaos.events { "event_id": "chaos_789",
"action_type": "kill_container", "target_service": "transaction-service",
"target_container": "transaction-svc-7d9f8-xk2p", "timestamp":
"2025-03-11T14:30:05Z" } Topic: anomaly.detected { "anomaly_id": "anom_101",
"service_name": "transaction-service", "metric_name": "cpu_usage",
"actual_value": 0.92, "expected_value": 0.45, "anomaly_score": 0.87, "timestamp":
"2025-03-11T14:30:10Z" } Topic: healing.actions { "action_id": "heal_202",
"anomaly_id": "anom_101", "action_type": "scale_up", "target_service":
"transaction-service", "replicas_before": 2, "replicas_after": 4, "success":
true, "recovery_time_seconds": 45, "timestamp": "2025-03-11T14:31:00Z" }
```

9.2 REST API Endpoints

Transaction Service (Port 8001) POST /api/v1/transactions - Create new transaction GET /api/v1/transactions/{id} - Get transaction details GET /health - Health check GET /metrics - Prometheus metrics **Chaos Agent (Port 8004)** POST /api/v1/chaos/trigger - Manually trigger chaos action GET /api/v1/chaos/events - List recent chaos events GET /api/v1/chaos/config - Get current chaos configuration PUT /api/v1/chaos/config - Update chaos settings (frequency, probability) **Anomaly Detector (Port 8005)** GET /api/v1/anomalies - List detected anomalies GET /api/v1/predictions - Get failure predictions GET /api/v1/model/stats - Model performance metrics **Self-Healing Agent (Port 8006)** GET /api/v1/healing/history - List past healing actions GET /api/v1/healing/strategies - Get learned strategies (Q-table) POST /api/v1/healing/manual - Manually trigger healing action

10. Timeline & Milestones

Week	Phase	Deliverables	Deployment
1-2	Base Services	Transaction, Risk, Notification services + Kafka	Docker Compose
2	Monitoring	Prometheus + Grafana + metrics dashboards	Docker Compose
3	Chaos Engineering	Chaos Agent + failure injection	Docker Compose
4	ML Detection	Anomaly Detector + Isolation Forest/Prophet	Docker Compose
5	Self-Healing	Healing Agent + automated recovery	Docker Compose
6	K8s Migration	Migrate to Kubernetes (Minikube)	Kubernetes
7	Polish	Benchmarks, documentation, demo video	Both

Key Milestones:

✓ Week 2: End-to-end transaction flow working on Docker

✓ Week 3: Chaos agent successfully kills services, they recover

✓ Week 4: ML model detects anomalies with < 15% false positive rate

✓ Week 5: Self-healing demonstrates 80%+ success rate, MTTR < 2 min

✓ Week 6: Entire system running on Kubernetes

✓ Week 7: Demo video complete, GitHub repo polished

11. Expected Outcomes

11.1 Technical Skills Acquired

Distributed Systems: • Event-driven architecture design • Service communication patterns (pub/sub, point-to-point) • Failure modes in microservices (cascading failures, thundering herd, split brain) • CAP theorem trade-offs in practice **Infrastructure & DevOps:** • Docker containerization (Dockerfile best practices, multi-stage builds) • Docker Compose orchestration (networks, volumes, depends_on) • Kubernetes fundamentals (Pods, Deployments, Services, ConfigMaps, Secrets) • Kubernetes API programmatic control (kubernetes-python library) • Container networking (bridge networks, overlay networks, service discovery) **Chaos Engineering:** • Failure injection techniques (kill, latency, resource exhaustion) • Observing system behavior under stress • Resilience patterns (circuit breakers, bulkheads, timeouts) • Chaos experimentation methodology **Machine Learning:** • Time-series anomaly detection (Isolation Forest, Prophet, LSTM Autoencoders) • Feature engineering for system metrics • Model evaluation (precision, recall, F1, ROC-AUC) • Production ML deployment (serving models, monitoring drift) **Observability:** • Metrics collection (Prometheus) • Dashboard design (Grafana) • Alert configuration (Prometheus Alertmanager) • Log aggregation best practices **Backend Development:** • Async Python programming (FastAPI, asyncio) • Kafka producer/consumer patterns • PostgreSQL schema design for transactional data • REST API design (OpenAPI/Swagger)

11.2 Resume Impact

Project Bullet Point (Before): "Built microservices application with Python and Docker" **Project Bullet Point (After):** "**ChaosMend: ML-Driven Self-Healing Microservices Platform** • Architected event-driven payment processing system handling 1000+ transactions/second with Kafka, PostgreSQL, and FastAPI • Implemented chaos engineering agent that randomly injects failures (pod kills, latency, resource exhaustion) to discover resilience gaps • Developed ML-based anomaly detection using Isolation Forest and LSTM autoencoders, achieving 90% accuracy with <10% false positives • Built self-healing agent with reinforcement learning that reduces Mean Time To Recovery (MTTR) from 15 minutes (manual) to <2 minutes (automated), achieving 85% healing success rate • Deployed on Docker Compose and migrated to Kubernetes with Prometheus/Grafana observability • **Technologies:** Python, FastAPI, Kafka, Docker, Kubernetes, PostgreSQL, Prometheus, Grafana, Scikit-learn, Prophet"

11.3 Interview Preparation

System Design Questions You Can Now Answer: Q: "Design a highly available payment processing system" A: "I actually built one for my ChaosMend project. Let me walk through the architecture..." Q: "How do you ensure reliability in distributed systems?" A: "In my project, I used chaos engineering to intentionally break services and observed failure modes. I found that..." Q: "How would you detect and prevent cascading failures?" A: "I implemented ML-based anomaly detection that monitors service metrics. When it detects patterns like increasing error rates in Service A followed by rising CPU in Service B, it..." Q: "Explain a time you debugged a production issue" A: "During chaos testing, I discovered that when Kafka consumer lag exceeded 1000 messages, the notification service would crash loop. I solved it by..." **Behavioral Questions You Can Answer:** Q: "Tell me about a challenging technical project" A: "ChaosMend was challenging because I had to learn Kubernetes, implement ML models, AND understand distributed systems failure modes simultaneously. Here's how I approached it..." Q: "Describe a time you learned a new technology quickly" A: "I'd never used Kubernetes before this project. Within 3 weeks, I was deploying multi-service applications, implementing auto-scaling, and controlling the K8s API programmatically..."

12. Getting Started

Prerequisites: • macOS with M4 chip (or any Mac with Apple Silicon) • Homebrew package manager installed • 16GB RAM minimum (8GB might work, 32GB ideal) • 50GB free disk space **Initial Setup Commands:** 1. Install Dependencies: brew install docker brew install python@3.11 brew install node brew install kubectl brew install minikube 2. Clone/Create Project: mkdir ChaosMend cd ChaosMend git init 3. Create Virtual Environment: python3 -m venv venv source venv/bin/activate pip install -r requirements.txt 4. Start Infrastructure: cd infrastructure/docker docker-compose up -d 5. Verify Setup: docker ps # Should see Kafka, Zookeeper, PostgreSQL open http://localhost:8080 # Kafka UI open http://localhost:3000 # Grafana (after adding to docker-compose) **Next Steps:** Week 1 begins with building the Transaction Service. Detailed step-by-step instructions will be provided for each phase, with regular checkpoints to ensure understanding before moving forward. **Support Resources:** • GitHub Issues: For tracking bugs and questions • Documentation: All design decisions documented in docs/ folder • Code Comments: Minimal but strategic comments in code • Demo Videos: Screen recordings showing each feature working **Success Mindset:** This project is designed for learning, not perfection. If something doesn't work on the first try, that's normal. Debugging distributed systems is a core skill. Each failure is an opportunity to understand the system more deeply.

Ready to build? Let's start with Week 1: Base Services.